Nutrition-Aware Indian Food Recognition Using YOLO11s and Conservative Class-to-Database Mapping

Jaspreet Singh*, Inderpal Singh†
*† Department of Computer Science and Engineering
National Institute of Technology Srinagar
Kashmir 190006, India
Email: [student email]

***Abstract*—Food recognition is useful only when visual predictions can be connected to nutrition information that users can understand and act on. This paper presents an end-to-end Indian food recognition system that combines YOLO-based object detection, a curated 72-class Indian food dataset, and a nutrition mapping layer derived primarily from the Indian Nutrient Database. The central engineering problem is not only detecting food objects, but also resolving inconsistent food labels, regional dish names, and incomplete nutrition database coverage without producing misleading macro estimates. Five YOLO-format food datasets were merged into a canonical dataset containing 48,693 exported image-label pairs. A visual verification procedure confirmed that all 72 classes have actual image samples and identified three sample-level annotation issues. The final YOLO11s detector achieved 0.830 mAP@50 and 0.692 mAP@50:95 on validation, and 0.820 mAP@50 and 0.680 mAP@50:95 on a held-out test split. The nutrition pipeline produces 1,010 food records in SQLite, including five verified supplemental rows for dishes absent from INDB, and creates 103 precomputed detector-to-database mappings. For the expanded dataset, all 72 classes are mapped to safe nutrition entries with source metadata. The implemented FastAPI and Next.js prototype demonstrates a practical workflow in which uploaded food images are processed by a YOLO detector, matched to nutrition records, and converted into calorie and macronutrient feedback.**

***Index Terms*—food recognition, object detection, YOLO, Indian food dataset, nutrition mapping, dietary recommendation, computer vision**

I. Introduction

Image-based food logging promises a lower-friction alternative to manual diet tracking. A user can photograph a meal and receive a list of detected food items, serving estimates, and nutrition values. In practice, however, the problem is more brittle than ordinary object detection. Food is deformable, frequently occluded, and often visually ambiguous. Indian food adds another layer of difficulty: dishes vary by region, share ingredients and colors, and are named inconsistently across datasets. A class such as `rice` may appear as `WhiteRice`, `satham`, or `boiled rice`; chutneys may be visible only as small side portions; and biryani variants can differ by ingredients rather than global shape.

The project described in this paper was built around a simple premise: a food recognition system should not stop at labels. A detected label must be mapped to a nutrition entry before it becomes useful. That mapping step is a source of real error. A fuzzy string match may return a food with a similar name but a different macro profile. For example, a detector class for `palak_paneer` should map to spinach paneer, not to a generic paneer preparation. Similarly, a class with no suitable INDB entry should use a verified supplemental row or remain unavailable rather than receive an arbitrary substitute.

This paper makes three contributions. First, it describes a dataset engineering pipeline that merges five YOLO-format Indian food datasets into a 72-class canonical dataset. Second, it presents a conservative class-to-database mapping strategy that prioritizes explicit semantic mappings and source-backed supplemental rows before fuzzy matching. Third, it reports a working full-stack prototype that links detection, nutrition lookup, macro calculation, and user-facing recommendations.

The work is best understood as an applied system paper. It does not claim a new detector architecture. Instead, it addresses the glue code, data cleaning, and mapping decisions needed to make an object detector useful in a nutrition-aware application. These decisions are often left implicit, but in this domain they determine whether the system returns helpful information or plausible-looking misinformation.

II. Related Work

Food computing has been studied as a broad field connecting image analysis, dietary assessment, recommendation, and human behavior [1]. Early food recognition systems such as FoodCam showed that smartphone-based food recognition could support real-time logging workflows [2]. Im2Calories framed the problem as a mobile vision food diary and highlighted the difficulty of estimating food energy from images alone [3]. These systems motivate the present work, but they also show why recognition must be connected to a reliable nutrition layer.

Object detection methods are well suited to meal images because a single image may contain multiple food items. Two-stage detectors such as Faster R-CNN achieve strong detection performance by separating proposal generation and classification [4]. One-stage detectors such as YOLO instead formulate

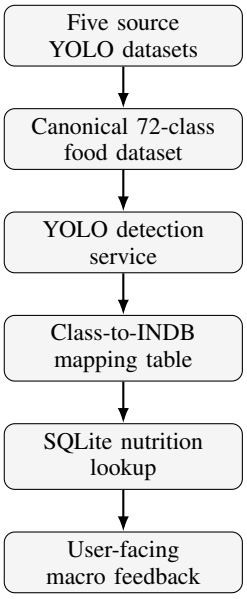

Fig. 1. Pipeline from source datasets to nutrition-aware user feedback.

detection as a direct regression and classification problem, which makes them attractive for interactive applications [5]. Later YOLO variants improved multi-scale prediction and practical deployment [6]. The implementation in this work uses Ultralytics YOLO tooling because it supports the YOLO annotation format used by the collected datasets and integrates cleanly with Python services [7].

Dataset quality remains a limiting factor. General object detection benchmarks such as COCO provide standardized annotations and category definitions [8]. Regional food datasets rarely have that level of consistency. The source data used in this project contained overlapping dishes, spelling variants, regional synonyms, and different class naming conventions. Nutrition data introduces a second schema mismatch: detector labels are short machine-learning classes, whereas nutrition databases store descriptive dish names with preparation details. The mapping layer in this work is therefore treated as part of the model pipeline rather than as a simple lookup table.

III. System Overview

Figure 1 summarizes the implemented workflow. Source datasets are normalized into a canonical YOLO dataset. A YOLO detector processes uploaded images and returns bounding boxes with class labels. The backend then resolves each class through a precomputed mapping table. When a safe mapping exists, nutrition values are returned from SQLite. When no safe mapping exists, the service reports the missing nutrition value instead of fabricating one.

The backend is implemented with FastAPI, PyTorch, Ultralytics YOLO, Pillow, Pandas, OpenPyXL, SQLite, and AioSQLite. The frontend is implemented with Next.js, React, TypeScript, Tailwind CSS, and Lucide icons. The data layer contains the merged YOLO dataset, the INDB spreadsheet, the generated SQLite database, and detector weights. The deployed model metadata currently contains a smaller legacy class list, while the generated 72-class dataset is retained as the expanded training corpus.

TABLE I
Generated dataset split summary.

Split	Image-label pairs	Role
Train	36,852	Training and augmentation
Validation	5,922	Model selection
Test	5,919	Held-out testing
Total	48,693	Complete export

IV. Dataset Construction

The dataset pipeline was designed to produce stable class names before model training. Five YOLO-format source datasets were scanned for image and label folders. Labels were parsed in the standard YOLO form:

$$y = (c, x, y, w, h), \tag{1}$$

where c is the class index and (x, y, w, h) are normalized bounding-box coordinates. Each source class name was normalized into a canonical label through lowercasing, punctuation removal, synonym handling, and manual regional-name mappings.

Table I reports the generated split sizes. The final export contains 48,693 image-label pairs. Training data received augmentation, while validation and test splits were kept separate to avoid contaminating evaluation data.

The canonical dataset contains 72 food classes. It includes rice dishes, curries, breads, chutneys, snacks, desserts, beverages, and non-vegetarian dishes. Examples include `aloo_gobi`, `coconut_chutney`, `fish_curry`, `medu_vada`, `palak_paneer`, `sambar_satham`, and `veg_briyani`. The merge process also produced machine-readable artifacts such as `data.yaml`, build logs, and split count tables.

A. Class Canonicalization

The class normalization step was kept explicit because food-name variation is not a minor formatting problem. The same dish may appear with English spellings, regional names, camel-case names, or source-specific abbreviations. If these labels are not merged, the detector learns several partial classes for the same dish and the nutrition layer later receives inconsistent inputs.

Table II gives examples of raw label variants and their canonical forms. The mapping dictionary was deliberately written in readable form so that a developer can audit each change. This is useful in food datasets because automatic string normalization alone cannot distinguish synonyms from visually related but distinct dishes.

The canonicalization pipeline also preserves multi-label images. If a source image contains more than one food object, each valid YOLO row is converted independently and written back to the exported label file. This avoids turning multi-dish

TABLE II
EXAMPLES OF SOURCE-LABEL CANONICALIZATION.

Raw or source-side variants	Canonical class
AlooGobi, aloo gobhi, aloo_gobi	aloo_gobi
CoconutChutney, thengai chutney	coconut_chutney
WhiteRice, satham, rice	rice
medu vadai, medu vada	medu_vada
lemon satham, lemon rice	lemon_rice

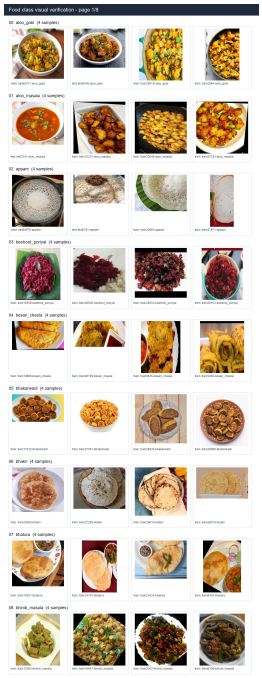

Fig. 2. Example visual verification sheet generated from actual dataset images.

meal images into single-label examples and keeps the object detection formulation intact.

B. Visual Class Verification

Class names were checked against actual images rather than accepted only from metadata. A verification script grouped exported images by canonical class suffix, sampled four images per class, parsed matching label files, and generated contact sheets. This process confirmed that all 72 classes have image samples and that the class list is aligned with the exported dataset.

The review found three sample-level annotation issues: one file labeled as `jalebi` that does not visually resemble jalebi, one `kaara_chutney` bounding box placed on a vada rather than the chutney, and one `nandu_masala` label with an extremely thin bounding box. These issues are important, but they do not indicate a class-order failure. They point to local annotation cleanup work for a later dataset revision.

TABLE III
EXAMPLES OF DETECTOR-CLASS TO NUTRITION-DATABASE MAPPINGS.

Class	Database food name	Score
aloo_gobi	Potato cauliflower (Aloo gobhi)	1.00
palak_paneer	Spinach paneer (Palak paneer)	1.00
rice	Boiled rice (Uble chawal)	1.00
chicken_biryani	Chicken pulao	0.80
ven_pongal	Plain khitchdi	0.75

V. NUTRITION MAPPING

The nutrition layer imports the Indian Nutrient Database spreadsheet into SQLite [9]. The final table contains 1,010 food records with calories, protein, carbohydrates, fat, and source metadata. Five rows are verified supplemental entries for dishes that are not present in the available INDB sheet. A second table, `yolo_mappings`, stores precomputed mappings from detector classes to nutrition database names.

The mapping algorithm follows a conservative order:

1) normalize the detector class name;
2) check an explicit manual mapping dictionary;
3) check exact normalized database names;
4) use a verified supplemental row when INDB has no safe row;
5) use fuzzy matching only above a threshold;
6) leave the class unavailable if no safe row exists.

This order matters. Manual mappings catch semantic cases that string matching handles poorly. For example, `aloo_gobi` maps to `Potato cauliflower (Aloo gobhi)`, and `palak_paneer` maps to `Spinach paneer (Palak paneer)`. These are not merely similar strings; they are dish-level matches.

The mapping decision for a detector class c can be written as:

$$M(c) = \begin{cases} M_{\text{manual}}(c), & \text{if manually verified,} \\ M_{\text{exact}}(c), & \text{if normalized names match,} \\ M_{\text{sup}}(c), & \text{if a verified supplemental row exists,} \\ M_{\text{fuzzy}}(c), & \text{if score}(c) \geq 0.78, \\ \emptyset, & \text{otherwise.} \end{cases} \tag{2}$$

The final case is intentional. In a nutrition system, the cost of a wrong confident mapping can be higher than the cost of an absent mapping. A missing or supplemental mapping can be surfaced to the user with provenance; a wrong mapping silently corrupts calorie and macro feedback.

The final database contains 103 YOLO-to-database mappings. This number is larger than the 72 expanded classes because the table also includes legacy deployed-model labels for backward compatibility. For the expanded dataset alone, 72 of 72 classes are mapped, giving a class coverage of 100.00%. The five classes absent from INDB are `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala`; they map to source-backed supplemental rows rather than weak fuzzy substitutes [10], [11], [12], [13], [14].

The lower-confidence mappings in Table III are intentionally recorded with scores. They represent nearest safe

Fig. 3. Prototype output showing detected food items with bounding boxes and nutrition-linked results.

TABLE IV
SYSTEM AND DETECTOR VALIDATION SUMMARY.

Validation item	Result
Canonical classes	72
Exported image-label pairs	48,693
Classes with visual samples	72 / 72
Sample-level annotation issues found	3
Validation mAP@50 / mAP@50:95	0.830 / 0.692
Test mAP@50 / mAP@50:95	0.820 / 0.680
Nutrition food records	1,010
Total YOLO-to-database mappings	103
Expanded-class safe mappings	72 / 72
Expanded-class mapping coverage	100.00%

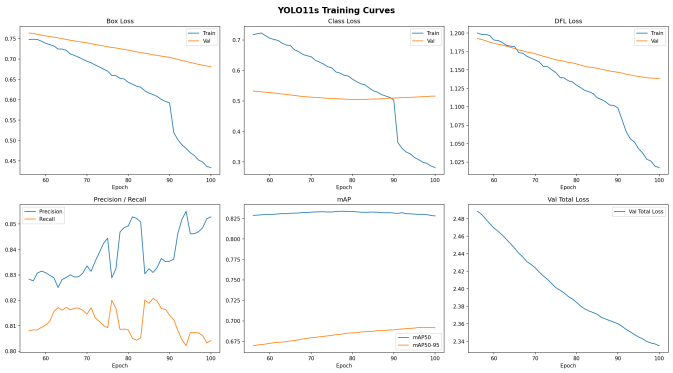

Fig. 4. Final YOLO11s training curves for the 72-class detector.

substitutes rather than exact dish equality. This makes uncertainty visible to developers and avoids presenting approximate matches as ground truth.

VI. Application Prototype

The runtime API accepts an image upload, runs YOLO inference, formats bounding boxes, and enriches each detection with nutrition values when available. The recommendation service uses a target calorie value, dietary goal, and optional health condition to produce macro guidance. The frontend displays detections, nutrition details, serving controls, and recommendations in a single workflow.

The design keeps detection and nutrition services separate. This separation avoids coupling model inference to database logic and makes it easier to replace either component. The mapping table also allows nutrition lookup to remain fast at runtime because expensive normalization and fuzzy matching are done during database build time rather than per request.

VII. Evaluation

The evaluation covers dataset integrity, detector performance, nutrition mapping coverage, and service-level behavior. The final model was trained with YOLO11s on 640-pixel images and batch size 16. Training resumed from epoch 56 and continued to 100 total epochs on a Tesla T4 GPU. The resumed stage completed in 10.636 hours. The best validation mAP@50 recorded in the training log was 0.83379 at epoch 78.

The best exported model was evaluated on both validation and held-out test splits. On validation, it achieved 0.847 precision, 0.807 recall, 0.830 mAP@50, and 0.692 mAP@50:95 across 5,922 images and 8,812 instances. On the test split, it achieved 0.842 precision, 0.801 recall, 0.820 mAP@50, and 0.680 mAP@50:95 across 5,919 images and 8,875 instances. The test mAP is close to the validation mAP, indicating that the detector is not only fitting the validation split. The model still shows uneven per-class behavior, which is expected for visually similar Indian dishes and classes with fewer unique source images.

Several checks were also performed against the runtime lookup path. Representative classes such as `aloo_gobi`, `palak_paneer`, `rice`, `AlooGobi`, and `WhiteRice` returned the expected nutrition display names after the mapping revision. This confirmed that the updated pipeline supports both expanded dataset labels and legacy class names.

A. Runtime Behavior Checks

The system was tested at the service boundary because the user-facing behavior depends on more than the database contents. A successful response must carry the detector label, display name, confidence, bounding box, and nutrition values in a shape that the frontend can render. It must also fail gracefully when an image contains no recognized food or when a class lacks a nutrition mapping.

These checks are small, but they guard against regressions that ordinary model training metrics would miss. A detector can still produce a label while the application returns the wrong database row; conversely, the nutrition service can be

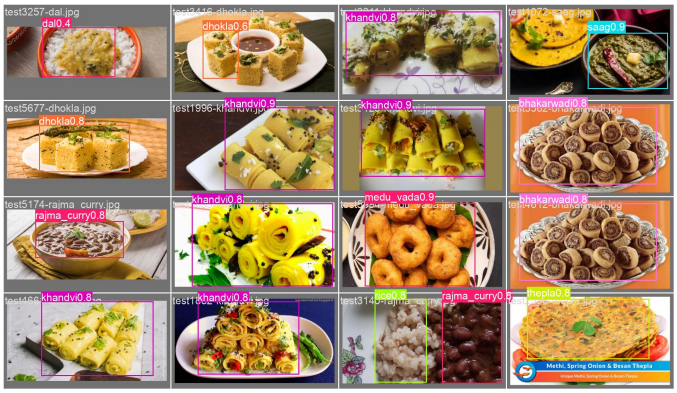

Fig. 5. Example YOLO11s detections from the held-out test split.

TABLE V
RUNTIME LOOKUP BEHAVIOR CHECKED DURING VALIDATION.

Input or condition	Expected behavior
`aloo_gobi`	Return potato cauliflower nutrition row
`palak_paneer`	Return spinach paneer nutrition row
`WhiteRice`	Resolve legacy label to boiled rice
No detected food	Return no-food response without nutrition values
Unmapped class	Preserve detection and mark nutrition unavailable

correct while the frontend receives fields under unexpected names.

VIII. Discussion

The most important finding is that nutrition-aware food recognition depends on decisions outside the detector. Object detection gives a label and a location. A health-oriented application must decide what that label means nutritionally. When class names and database rows do not match exactly, the mapping layer becomes a source of domain error.

The conservative approach used here is useful because it avoids a common failure mode: returning a confident nutrition result for the wrong food. The five classes absent from INDB are handled through verified supplemental rows instead of weak substitutes. This is especially important for sweets, snacks, and seafood dishes where calories and macronutrient profiles vary widely by preparation.

The visual verification step also proved necessary. A metadata-only check would have confirmed class names and counts, but it would not have found the incorrect `jalebi` sample, misplaced `kaara_chutney` box, or thin `nandu_masala` annotation. The process is inexpensive and should be part of future dataset build runs.

Another practical observation is that backward compatibility matters during dataset expansion. The expanded dataset contains 72 classes, while the deployed model metadata still contains a smaller legacy class list. Rebuilding the mapping table only for the new dataset would break old model labels. The database builder therefore loads the expanded dataset classes first and then appends legacy labels. This keeps the application usable while the larger model is still being trained or evaluated.

IX. Threats to Validity

The reported coverage numbers measure mapping and data integrity, not final dietary accuracy. A mapped class can still have portion-size uncertainty, and a database entry can differ from the exact recipe in the image. Indian dishes vary strongly by oil quantity, ingredient ratios, and regional preparation. For that reason, the system should be interpreted as a decision-support prototype rather than a clinical nutrition instrument.

The visual verification procedure is also sample based. Four examples per class are enough to find obvious class-order errors and some annotation problems, but they do not prove that every label in the dataset is correct. A future audit should combine contact-sheet review with automated checks for invalid boxes, empty crops, duplicate labels, and anomalous image-label pairs.

X. Limitations and Future Work

The current system has four main limitations. First, aggregate detector metrics are available, but deeper per-class error analysis is still needed to prioritize the weakest food categories. Second, portion estimation is not solved; nutrition values are returned per database serving basis and need user adjustment. Third, five classes rely on supplemental nutrition sources because the available INDB sheet is incomplete for those dishes. Fourth, the visual verification process found annotation noise that should be corrected before the next training cycle.

Future work should replace or cross-check supplemental nutrition rows with official Indian food composition entries, improve weak detector classes using targeted data collection, and add portion estimation using either depth cues, reference objects, or calibrated serving-size inputs. A review dashboard for class-wise annotation inspection would also make dataset maintenance easier.

XI. Conclusion

This paper presented a practical nutrition-aware Indian food recognition system built around YOLO detection, dataset normalization, and conservative nutrition mapping. The work merged five YOLO-format datasets into a 72-class canonical dataset with 48,693 exported image-label pairs, verified the class list through actual image samples, produced 1,010 nutrition records, and mapped all 72 expanded classes to safe database entries. The resulting prototype demonstrates that regional food recognition requires more than a detector: it requires careful class naming, transparent database mapping, source provenance, and explicit handling of uncertainty. These engineering choices make the difference between a visually impressive demo and a system that can support credible dietary feedback.

ACKNOWLEDGMENT

The implementation and dataset engineering artifacts used in this manuscript were developed as part of a final year project in the Department of Computer Science and Engineering, National Institute of Technology Srinagar.

REFERENCES

[1] W. Min, S. Jiang, L. Liu, Y. Rui, and R. Jain, "A survey on food computing," *ACM Computing Surveys*, vol. 52, no. 5, pp. 1–36, 2019.

[2] Y. Kawano and K. Yanai, "Foodcam: A real-time food recognition system on a smartphone," *Multimedia Tools and Applications*, vol. 74, pp. 5263–5287, 2014.

[3] A. Meyers, N. Johnston, V. Rathod, A. Korattikara, A. Gorban, N. Silberman, S. Guadarrama, G. Papandreou, J. Huang, and K. P. Murphy, "Im2calories: Towards an automated mobile vision food diary," *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1233–1241, 2015.

[4] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 779–788.

[6] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[7] Ultralytics, "Ultralytics yolo documentation," 2024, software documentation; accessed during project development.

[8] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision*. Springer, 2014, pp. 740–755.

[9] Indian Nutrient Database, "Indian nutrient database spreadsheet used in project data layer," 2026, pLACEHOLDER BIBTEX ENTRY: replace with the official publication or institutional citation for INDB.xlsx before final submission.

[10] FatSecret India, "Evolve bhakarwadi nutrition facts per 100 g," https://www.fatsecret.co.in/calories-nutrition/evolve/bhakarwadi/100g, 2026, accessed 2026-06-06.

[11] Clearcals, "Ghevar recipe calories and nutrition per 100 g," https://clearcals.com/recipes/ghevar/, 2026, accessed 2026-06-06.

[12] ——, "Jalebi recipe calories and nutrition per 100 g," https://clearcals.com/recipes/jalebi/, 2026, accessed 2026-06-06.

[13] ——, "Khandvi recipe calories and nutrition per 100 g," https://clearcals.com/recipes/khandvi/, 2026, accessed 2026-06-06.

[14] ——, "Nandu kari recipe calories and nutrition per 100 g," https://clearcals.com/recipes/nandu-kari/, 2026, accessed 2026-06-06.